

Отчет по 3 этапу внешнего курса

Внешний курс этап 3

Богомолова Полина Петровна

2026-05-09

Богомолова Полина Петровна Студент, ФФМиЕН10322535621032253562@rudn.ru Российский университет дружбы народов

Цель работы

Получить практические и теоретические знания и умения по работе с Linux

Задание

Выполнить все задания 3 этапа внешнего курса

Вопрос 1

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

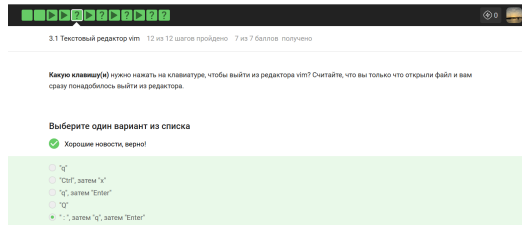


Рисунок 1: Вопрос 1

Выбран вариант “:”, затем “q”, затем “Enter”. После открытия vim в обычном режиме, для выхода нужен переход в командный режим через двоеточие и ввод q. Нажатие “q” без двоеточия запускает запись макроса. Ctrl+x работает в nano, но не в vim. Заглавная Q переводит в режим Ex.

Вопрос 3

Строку “one two three four five” нужно преобразовать в “three four four four five”. Какие наборы нажатий выполнят такое редактирование?

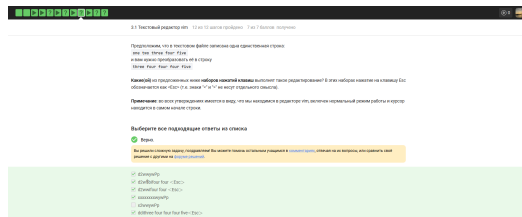


Рисунок 3: Вопрос 3

Верно: d2wwwyWp (удаляет два слова и вставляет four), d2w\$bifour four (удаляет начало и дописывает текст), d2wwifour four (аналогично через режим вставки), xxxxxxxxwyWp (посимвольное удаление восьми знаков), ddithree four four four five (полная очистка и набор заново). Неверно: x2wwwyWp (удалён только один символ).

Вопрос 4

Заменить во всём файле только первое вхождение Windows на Linux в каждой строке.
Укажите команду полностью.



Рисунок 4: Вопрос 4

`:%s/Windows/Linux/` — символ `%` для всего файла, отсутствие `g` заменяет только первое вхождение. `:s/...` — только текущая строка. `:%s/.../g` — все вхождения. `:%s/windows/Linux/` не сработает из-за регистра.

Ознакомьтесь с режимом выделения (Visual) в vim. Отметьте все верные утверждения.

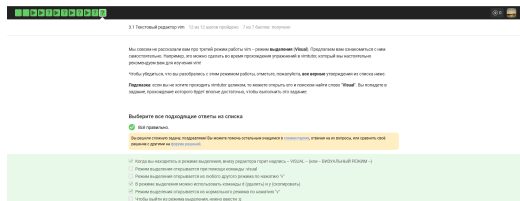


Рисунок 5: Вопрос 5

Верно: внизу отображается – VISUAL –, d и у работают на выделенном блоке, вход по v из нормального режима. Неверно: команды :visual нет, v из другого режима не сработает без Esc, выход через :q закроет редактор, а не режим.

Вопрос 6

Вы в `bash` набрали `A1`, `A2`, `A3`, запустили `sh` (`B1`, `B2`, `B3`), затем `bash` (`C1`, `C2`, `C3`). Какие команды появятся в истории по стрелкам?

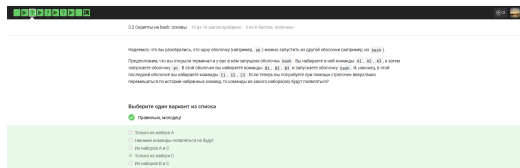


Рисунок 6: Вопрос 6

Только из набора C. Каждая оболочка хранит историю в памяти, дочерний `bash` не видит команды родительских процессов. Наборы A и B остались в памяти предыдущих оболочек. Набор B принадлежит `sh`, у которой своя история.

Вопрос 7

Скрипт запущен в `/home/bi/Documents/`:

```
#!/bin/bash
```

```
cd /home/bi/
```

```
touch file1.txt
```

```
cd /home/bi/Desktop/
```

Абсолютный путь до `file1.txt`?

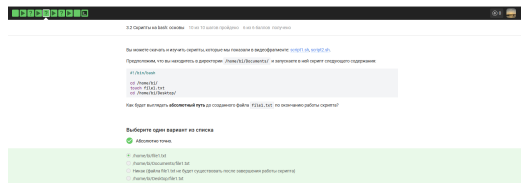


Рисунок 7: Вопрос 7

Какие строки могут быть именами переменных в bash?

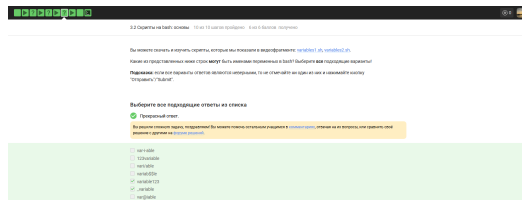


Рисунок 8: Вопрос 8

Верно: variable123 (буквы и цифры, цифра не в начале), _variable (подчёркивание разрешено). Неверно: var-i-able (дефис — оператор), 123variable (цифра в начале), vari/able (слэш для путей), variab\$\$le (доллар для подстановки), var@iable (символ не из набора).

Вопрос 9

Скрипт принимает два аргумента и выводит: Arguments are: \$1=первый_аргумент
\$2=второй_аргумент

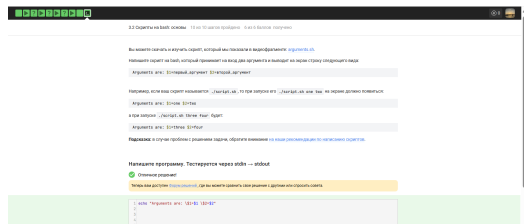


Рисунок 9: Вопрос 9

echo “Arguments are: \$1=\$1 \$2=\$2” — слэш экранирует первый доллар, выводя текст \$1, второй доллар подставляет значение. Без экранирования оба доллара раскроются. Одинарные кавычки не раскрывают переменные.

Вопрос 10

Впишите в if `[[ ... ]]` условие, которое всегда выводит “True”, независимо от переменных и аргументов.

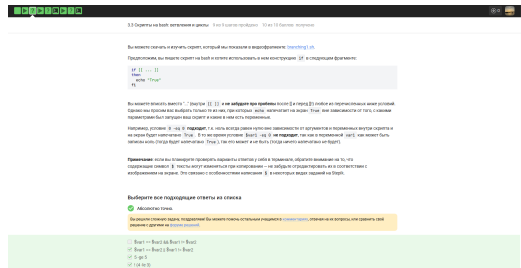


Рисунок 10: Вопрос 10

Всегда True: `$var1 == $var2 || $var1 != $var2` (покрывает все случаи), `5 -ge 5` (константа), `!(4 -le 3)` (отрицание ложного), `-z ""` (проверка пустой строки). Не всегда: `$var1 == $var2 && $var1 != $var2` (противоречие), `$# -gt 0` (зависит от аргументов).

Вопрос 11

Скрипт с var=3 и var=5:

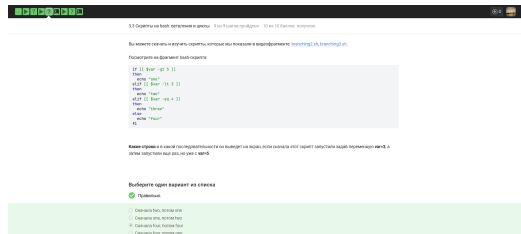
```
if [[ $var -gt 5 ]]; then echo "one"
```

```
elif [[ $var -lt 3 ]]; then echo "two"
```

```
elif [[ $var -eq 4 ]]; then echo "three"
```

```
else echo "four"; fi
```

Что выведется?



Вопрос 12

Скрипт принимает число студентов ($0 - \infty$) и выводит сообщение по правилам.

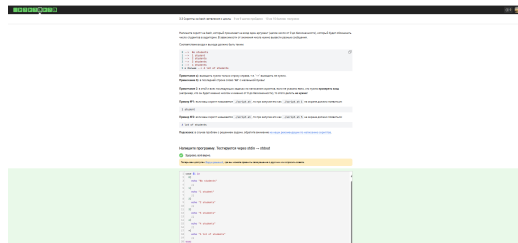


Рисунок 12: Вопрос 12

case \$1 in

0) echo "No students";;

1) echo "1 student";;

[2-4]) echo "\$1 students";;

Вопрос 13

```
for str in a , b , c_d; do
```

```
echo "start"
```

```
if [[ $str > "c" ]]; then continue; fi
```

```
echo "finish"
```

```
done
```

Сколько раз выведется start и finish?

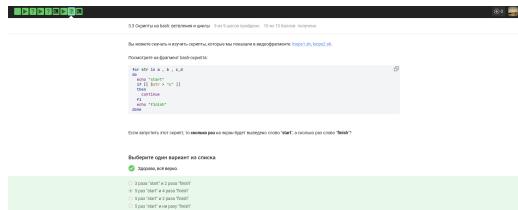


Рисунок 13: Вопрос 13

Вопрос 14

Скрипт: бесконечный опрос имени и возраста, определяет группу (child ≤ 16 , youth 17-25, adult >25). Пустое имя или возраст 0 — выход с “bye”.

Напишите скрипт на bash, который будет определять в какую возрастную группу попадает пользователь. При запуске скрипта должен вывести сообщение “enter your name:” и ждать от пользователя ввода имени (используя `read`), чтобы прочитать его). Когда имя введено, то скрипт должен написать “enter your age:” и ждать ввода возраста (опять же using `read`). Когда возраст введен, скрипт пишет на экран “Okim, your group is «группа»”, где «группа» определяется на основе возраста по следующим правилам:

- младше либо равно 16: “child”;
- от 17 до 25 (включительно): “youth”;
- старше 25: “adult”.

После этого скрипт опять выводит сообщение “enter your name:” и всё начинается по новой (бесконечный цикл). Если в какой-то момент работы скрипта будет введено пустое имя или возраст 0, то скрипт должен написать на экран “bye” и закончить свою работу (выход из цикла).

Примеры корректной работы скрипта:

№1:

```
./script.sh
enter your name:
Egor
enter your age:
16
Egor, your group is child
enter your name:
Elena
enter your age:
0
bye
```

№2:

```
./script.sh
enter your name:
Elena Petrovna
enter your age:
25
Elena Petrovna, your group is youth
enter your name:
bye
```

Подсказка: в случае проблем с решением задачи, обратитесь внимание на наши рекомендации по написанию скриптов.

Напишите программу. Тестируется через stdin → stdout

✓ Хорошие новости, верно!

Почему вам доступны [форумы решений](#), где вы можете проверить свои решения с другими или задать вопрос?

Вы решили данную задачу, поздравляем! Вы можете получить дополнительное [удовольствие](#) в [комментариях](#), ответить на их вопросы, или проверить свои решения с другими на [форумах решений](#).

```
1 | if [ -z "$(cat /dev/shm/flag.txt)" ]; then echo "You have solved the task."; fi
```

Какие инструкции увеличат a на b ($a=10, b=5 \rightarrow a=15$)?

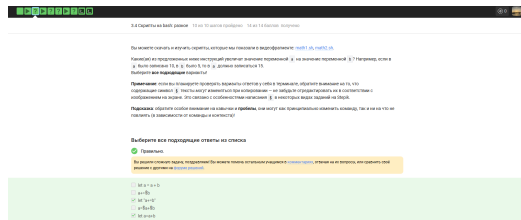


Рисунок 15: Вопрос 15

Верно: `let a = a + b` (арифметическое выражение),
`a+=b()`, `let "a+\" = b\"()\", let a = a + b()`. : `a =a+$b` (строковая конкатенация, результат “10+5”).

Вопрос 16

Скрипт в /home/bi/Documents/:

```
#!/bin/bash
```

```
cd /home/bi/
```

```
echo "pwd"
```

Что выведется?

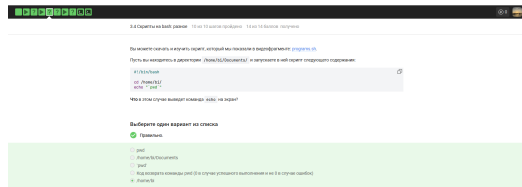


Рисунок 16: Вопрос 16

/home/bi. cd меняет директорию. Обратные кавычки выполняют pwd и подставляют

Вопрос 17

Как проверить код возврата program, которая пишет в stdout? Выберите верные конструкции if.

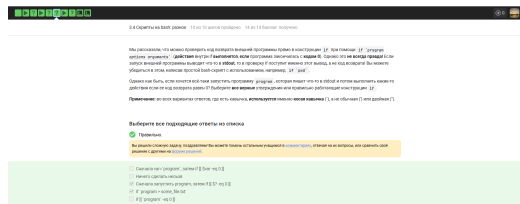


Рисунок 17: Вопрос 17

Верно: `program; if [[ ? — eq 0 ]]` (? хранит код возврата), `if program > some_file.txt` (перенаправление убирает stdout). Неверно: `var=program; if [[ $var -eq 0 ]]` (сравнивается вывод, а не код), `if [[ program -eq 0 ]]` (текст сравнивается с нулём).

Вопрос 18

```
counter () {  
  local let "c1+= $1"  
  let "c2+= ${1}*2"  
}
```

Вывод echo "counters are \$c1 and \$c2" после 10 вызовов с 1..10.

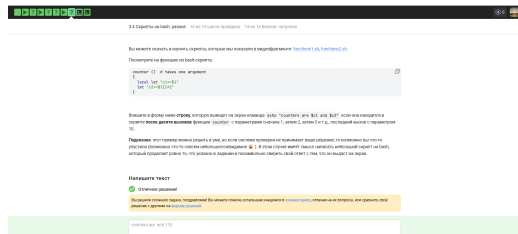


Рисунок 18: Вопрос 18

Вопрос 19

Скрипт для НОД (алгоритм Евклида). Ввод двух чисел, пустая строка — выход.

Примечание: в вызове функции из себя самой нет ничего страшного или неправильного, т.ч. смело вызывайте `gcd` прямо внутри `gcd`!

Примечание 2: для завершения работы функции в произвольном месте, можно использовать инструкцию `return` (все инструкции функции после `return` выполняться не будут). В отличие от `exit` эта команда завершит только функцию, а не выполнение всего скрипта целиком. Однако в данной задаче можно обойтись и без использования `return`!

Подсказка: в случае проблем с решением задачи, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Отличное решение!

Теперь вам доступны [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 # put your shell (bash) code here
2 gcd () {
3     N=$1
4     N=$2
5
6     if [[ $N -eq $N ]]; then
7         echo "gcd is $N"
8     elif [[ $N -gt $N ]]; then
9         gcd $((N - N)) $N
10    else
11        gcd $N $((N - N))
12    fi
13 }
14
15 while true; do
16     read n_in n_in
17
18
19     if [[ -z "$n_in" ]]; then
20         echo "bye"
21         break
22     fi
23
24     gcd $n_in $n_in
25 done
```

Рисунок 19: Вопрос 19

Калькулятор на bash. Команды: exit, операнд операция операнд (+,-,*,/,%,**), иначе error.

Напишите калькулятор на bash. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

1. Слово "exit". В этом случае скрипт должен вывести на экран слово "bye" и завершить работу.
2. Три аргумента через пробел – первый операнд (целое число), операция (одна из "+", "-", "*", "/", "%", "**") и второй операнд (целое число). В этом случае нужно проанализировать указанную операцию над заданными числами и вывести результат на экран. После этого перейти в режим ожидания новой команды.
3. Любая другая команда из одного аргумента или из трех аргументов, но с операцией на неопределенно. В этом случае нужно вывести на экран слово "error" и завершить работу.

Чтобы проверить работу скрипта, вы можете запустить сразу несколько команд в файл и передать его скрипту на stdin (т.е. выполнить: `./script.sh < input.txt`). В этом случае он должен вывести сразу все ответы на экран.

Например, если вводимый файл будет следующего содержания:

```
10 + 5
2 ** 10
exit
```

то на экране будет:

```
11
1024
bye
```

Если же на вход поступит следующий файл:

```
3 / 5
2/10
exit
```

то на экране будет:

```
0.6
error
```

т.к. вторая команда была некорректной (в ней всего один аргумент, т.е. нет пробелов между числами и операцией, а единственная допустимая команда из одного аргумента это "exit").

Подсказка: в случае проблем с решением задачи, обратитесь пожалуйста на наши [форумы](#) по написанию скриптов.

Напишите программу. Тестируется через stdin → stdout

✓ Абсолютно точно.

Получить код решения

```
1 # put your shell (bash) code here
2 while true
3 do
4     read num1 op num2 rest
5     if [[ $num2 == "exit" ]]
6     then
7         echo "bye"
8         exit
9     elif [[ ! $num1 =~ ^-[0-9]+$ || ! $num2 =~ ^-[0-9]+$ || ! $rest == "" ]]
10    then
11        echo "error"
```

Вопрос 21

В /home/bi файлы: Star_Wars.avi, star_trek_OST.mp3, STARS.txt, stardust.mpeg, Eddard_Stark_biography.txt. Какие найдет `find -iname "star"`, но не найдет `-name "star"`?

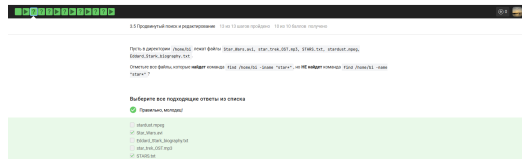


Рисунок 21: Вопрос 21

Star_Wars.avi и STARS.txt — `-iname` игнорирует регистр, `-name` ищет только строчные. star_trek_OST.mp3 и stardust.mpeg найдут обе команды. Eddard_Stark_biography.txt не начинается со star.

Верные утверждения про -path и -name команды find.

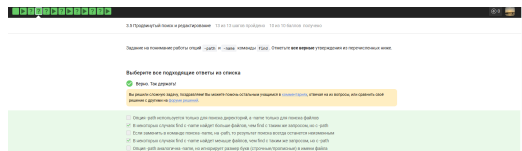


Рисунок 22: Вопрос 22

Верно: -name может найти больше или меньше, чем -path, так как -name ищет в имени, -path — во всём пути. Неверно: привязка к типу объекта (обе работают с файлами и папками), результат всегда одинаков (разная область поиска), -path игнорирует регистр (нужны -ipath и -iname).

Вопрос 23

Структура: /home/bi/dir1/file1, dir1/dir2/file2, dir1/dir2/dir3/file3.

`find /home/bi -mindepth 2 -maxdepth 3 -name "file*"`

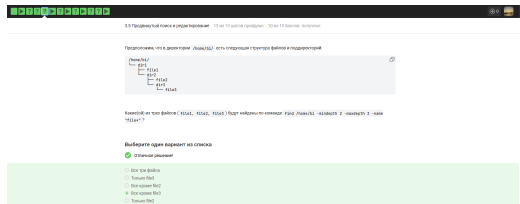


Рисунок 23: Вопрос 23

file1 на глубине 2, file2 на глубине 3 — оба в диапазоне. file3 на глубине 4, превышает maxdepth 3. Найдены file1 и file2.

Вопрос 24

Файл из 10 строк, в каждой “word”. Какая команда создаст наибольший results.txt?

grep “word”, grep -A 1, grep -B 1, grep -C 1.

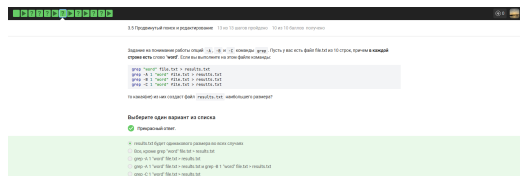


Рисунок 24: Вопрос 24

Все команды создадут файл одинакового размера. Все 10 строк уже содержат word, опции контекста не добавляют новых уникальных строк. В results.txt всегда 10 строк.

Какие строки выведет `grep -E "[xklXKL]?[uU]buntu$" text.txt`?

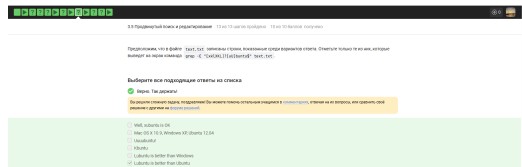


Рисунок 25: Вопрос 25

Kubuntu (K в наборе, Ubuntu в конце), Lubuntu is better than Ubuntu (заканчивается на Ubuntu). Не подходят: xubuntu в середине строки, строки с цифрами или знаками в конце, Windows в конце.

Что будет, если в `sed -n "[a-z]*/p" text.txt` убрать `-n`?

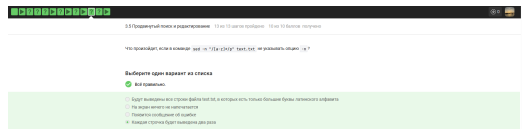


Рисунок 26: Вопрос 26

Каждая строка выведется дважды: автоматическая печать `sed` плюс команда `p`. Шаблон `[a-z]*` соответствует любому количеству строчных букв, включая ноль, поэтому совпадают все строки.

Вопрос 27

Инструкция sed, заменяющая “аббревиатуры” (≥2 заглавные буквы, окружённые пробелами) на “abbreviation” в input.txt, вывод в edited.txt.

Запишите в форму ниже инструкцию `sed`, которая заменит все “аббревиатуры” в файле `input.txt` на слово “abbreviation” и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что в инструкции должны быть указаны и сам `sed`, и оба файла!

Под “аббревиатурой” будем понимать слово, которое удовлетворяет следующим условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

При этом будем считать, что в тексте не может быть две “аббревиатуры” подряд. Например, текст “ YOU YOU and YOU!” является некорректным (в нем есть две “аббревиатуры”, но они идут подряд) и на таких примерах мы проверять вашу инструкцию не будем.

Пример: если у вас был текст “Hi, I heard these songs by ABBA, TLA and DM !”, то он должен быть преобразован в “Hi, I heard these songs by ABBA, abbreviation and abbreviation !”.

Примечание: после вашей замены “аббревиатуры” на слово “abbreviation” количество пробелов в тексте не должно меняться!

Внимание! Во время проверки мы не запускаем команду, которую вы ввели на реальном файле с “аббревиатурами” (это небезопасно, можно же ввести `rm -rf /*`). Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использован именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем запускаем её смысловую часть (т.е. поиск по регулярному выражению и замена на “abbreviation”) на тестовых примерах. К сожалению, наш запуск не идеально повторяет `sed`, но он очень близок к нему. Главная “несовместимость” заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. *, +, ? и {}). Однако эту “несовместимость” легко исправить указав при помощи “[” и “)” какой из символов к чему относится! Например, регулярное выражения `a+?` (ноль или один раз по одной или более букве “a”) нужно записать как `{a}+?` (при этом запись `{a}+?`, конечно же, не поможет).

Напишите текст

☒ Так точно!

```
sed 's/[A-Z][A-Z]\{2,\} / abbreviation /g' input.txt > edited.txt
```

Рисунок 27: Вопрос 27

Какую опцию указать в gnuplot, чтобы графики не закрывались?

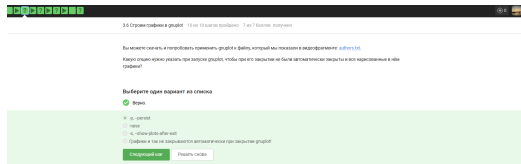


Рисунок 28: Вопрос 28

-p, -persist — оставляет окна открытыми после завершения gnuplot. Без опции окна закрываются. -raise управляет фокусом, а не сохранением.

Вопрос 29

data.csv с 10 числами в двух столбцах, без заголовков. Команды:

```
set key autotitle columnhead
```

```
plot 'data.csv' using 1:2
```

Название ряда и сколько точек?

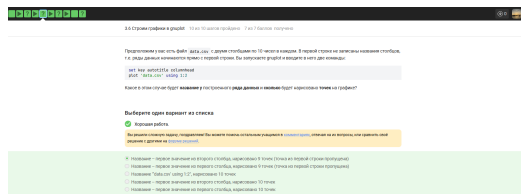


Рисунок 29: Вопрос 29

Название — первое значение из второго столбца. Первая строка стала заголовком и исключена. Точек — 9. 10 точек неверно, заголовок из первого столбца неверно.

Скрипт gnuplot: три деления на оси X в координатах x1, x2, x3 с подписями “point N, value”.

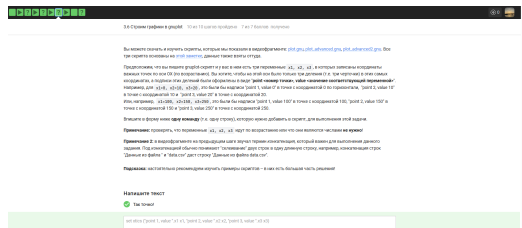


Рисунок 30: Вопрос 30

set xtics (“point 1, value”.x1 x1, “point 2, value”.x2 x2, “point 3, value”.x3 x3)

Каждая метка: текст через конкатенацию, затем координата. Скобки группируют список. Запятые разделяют метки.

Вопрос 31

Изменить move.rot: зеркально отразить по горизонтали, вращение в обратную сторону, в 2 раза быстрее.

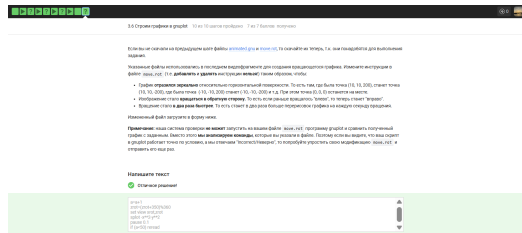


Рисунок 31: Вопрос 31

$a=a+1$

$zrot=(zrot+350)\%360$

set view xrot,zrot

Установить права `rw-rw-r--` (было `r--r--r--`). Выберите верные команды.

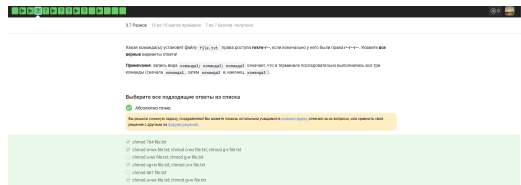


Рисунок 32: Вопрос 32

Верно: `chmod 764`, `chmod a+wx`; `chmod o-wx`; `chmod g-x`, `chmod ug+w`; `chmod u+x`, `chmod u+wx`; `chmod g+w`. Неверно: `chmod u-wx` (удаление прав), `chmod 467` (`r-rw-rwx`).

Директория `dir` (`root:root, rwxr-xr-x`). После какой команды `user` из `group group` сможет создать файл?

Предположим вы использовали команду `sudo` для создания директории `dir`. По умолчанию для `dir` были выставлены права доступа `rwxr-xr-x` (владелец `root`, группа `root`). Таким образом никто кроме пользователя `root` не может ничего записывать в эту директорию, например, не может создавать файлы в ней.

После выполнения какой команды `user` из группы `group` всё-таки сможет создать файл внутри `dir`? Укажите все верные варианты ответов!

Примечание: считаем, что все команды выполняются от имени `user`, если явно не указано, что команда выполнена с `sudo`.

Примечание 2: мы выбрали пример с директорией, а не с файлом не случайно.

Дело в том, что если создать при помощи `sudo` файл с правами `rw-r--r--` в директории, которая принадлежит пользователю, то возникнет любопытная ситуация. С одной стороны пользователь может удалить этот файл (т.к. ему разрешено удалять все файлы внутри его директории) и может прочитать его содержимое (т.к. право `r` у файла установлено для всех), с другой стороны он не может этот файл редактировать (т.к. право `w` у файла есть только для `root`). При этом некоторые "умные" редакторы, например, `vim` позволят даже редактировать этот файл, но сделают они это своеобразно: через удаление оригинала и создание копии уже с нужными правами (удалять мы можем, а раз можем читать, то и копию создать не сложно). Итого получается, что несмотря на права `rw-r--r--`, пользователь может сделать с этим файлом почти всё что угодно!

В случае же, когда речь идет о директории созданной `root`, ситуация будет проще: пользователь сможет смотреть её содержимое (у него есть право `r`), но удалять и создавать файлы в ней не сможет (права `w` у него нет).

Важно отметить, что директории в `Linux` это в каком-то смысле файлы. Содержимое такого "файла" – это записи о файлах и поддиректориях этой директории (грубо говоря их названия). Таким образом, право `r` у директории дает возможность просматривать "записи", т.е. просматривать её состав. Право `w` у директории дает возможность удалять/добавлять новые "записи", т.е. удалять/создавать файлы/поддиректории в ней.

На самом деле и это еще не всё. Существует так называемый [sticky bit](#) (атрибут файла или директории), выставление которого меняет описанное выше поведение. Файлы (или директории) с таким атрибутом сможет удалить только их владелец вне зависимости от прав, установленных у директории, в которой эти файлы (или директории) лежат!

Отдельное спасибо слушателю курса [Alexey Antipovsky](#) за помощь в оформлении **Примечания 2!**

Выберите все подходящие ответы из списка

☒ Хорошие новости, верно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ `sudo chown :group dir`
- ☒ `sudo chown user:group dir`
- ☐ `sudo chmod o+x dir`
- ☐ `sudo chmod g+w dir`

Какие характеристики файла считает `wc`?

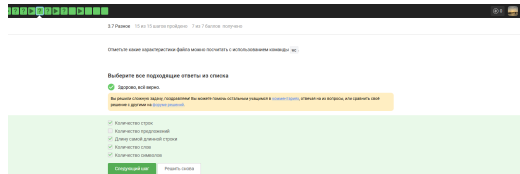


Рисунок 34: Вопрос 34

Строки (`-l`), длина самой длинной строки (`-L`), слова (`-w`), символы (`-m`) и байты (`-c`).
Предложения — нет, `wc` не распознаёт пунктуацию.

Команда для вывода размера текущей директории в удобном формате.

3.7 Раздел: 15 из 15 шагов пройдено 7 из 7 баллов получено

Поставьте в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (При этом **размер** нужно вывести в **добром для чтения формате** (например, вместо 3048 байт надо вывести 3,0к) и **больше** на экран выводить **ничего не нужно**). В команде **разрешено** только **использование** для выполнения задания **символов и аргументов**, **нижние** **символы** **разрешены** на экран.

Пример: если в текущей директории есть файл `foo` и две поддиректории в которых есть файлы `bar` и `baz`, то заданная команда должна вывести на экран одно число: `3,0к` (также на экран может быть выведен код и символ `^`, обозначающий, что это размер именно текущей директории).

Напишите текст

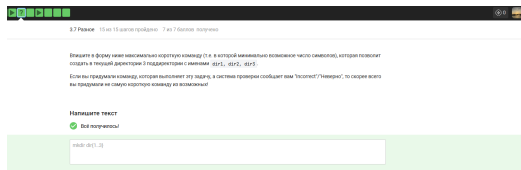
☒ Абсолютно точно.

Рисунок 35: Вопрос 35

`du -sh .`

`-s` — итоговая сумма без детализации. `-h` — человекочитаемый формат. Точка — текущая директория.

Максимально короткая команда для создания dir1, dir2, dir3.



The screenshot shows a web-based coding challenge interface. At the top, there's a progress bar indicating '3/7 Решено' (3/7 Solved) and '15 из 15 шагов пройдено' (15 of 15 steps completed). Below this, the problem statement is in Russian: 'Введите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами dir1, dir2, dir3.' (Enter in the form below the maximum short command (i.e. with the minimum possible number of symbols) that will allow you to create 3 subdirectories with names dir1, dir2, dir3 in the current directory). It also mentions a feedback system. Below the text, there's a section 'Напишите текст' (Write text) with a green checkmark and the text 'Всё получилось!' (Everything worked out!). At the bottom, there's a text input field containing the command 'mkdir dir{1..3}'.

Рисунок 36: Вопрос 36

`mkdir dir{1..3}`

Фигурные скобки раскрывают последовательность 1-3, подставляя в имя dir. Одна команда создаёт три директории.

Освоила Linux на более высоком уровне, научилась использовать полезные команды и программы.